U209: a joint decoder reduction for the consistency of Peano arithmetic

Comparing three 211-state constructions and combining their savings

Research prepared for Vladimir Reshetnikov • 29 October 2026

Abstract

The three supplied U211 archives contain different literal machines, all implementing the same inherited numerical proof search. Their individual reductions delete native states 180, 181, and 218, respectively, but their published decoder arrangements do not permit those patches to be overlaid. We give a new association of an 18-selector region containing 3,096 source primitives. It preserves the helper behavior required by the first two folds and makes every remaining entrance to state 218 carry a zero predecessor tag. All three deletions then coexist, producing an explicit binary, one-tape machine with **209 working states and 418 transitions**, including blank initialization. Its native and dense tables, deterministic builder, complete source, and replay programs accompany this report.

The new machine preserves every one of 3,909 primitive instructions and all 5,083 ordinal control edges. Ten exhaustive local cases prove the final three-fold simulation; two more cases prove the arithmetic-entry repair. A fresh symbolic derivation checks all 5,084 arbitrary-value primitive contracts and covers every transition row. The exact 48,140-step blank startup is unchanged. These results give an ordinary PA-formalizable reduction to the common U212 parent. Composing it with the explicitly retained numerical-calculus and U/PA translations gives the intended consistency endpoint. The deliverable is a mathematically justified *candidate with executable inductive evidence*, not a newly exported Lean, Rocq, MM0, or PA kernel certificate. No consistency, unconditional nonhalting, or global minimality assertion is made.

Contents

1 Result and exact scope **3**

2 What the three supplied proposals actually contain **4**

2.1 Pinned inventory and common parent 4

2.2 Machine and evidence comparison 4

2.3 A concrete documentation difference 5

3 Logical calibration retained by the new machine **5**

3.1 Unary U and its PA strength 5

3.2 The calculus actually executed 6

3.3 Relation to the supplied ZFC formalizations 6

4 The new common source arrangement **7**

4.1 Why simply combining the published patches fails 7

4.2 The fixed 18-selector allocation 7

4.3 The arithmetic-entry repair 8

4.4 Exact finite source evidence 8

5 Three state deletions with complete local proofs 8

5.1 The predecessor invariants 8

5.2 The two tagged folds 9

5.3 Deleting 218 by an existing behavior 9

5.4 A direct certificate for the combined result 10

6 Fresh unbounded literal-runtime verification 10

6.1 The quantified whole-tape frame 10

6.2 Sound elementary proof rules 11

6.3 Primitive endpoints and source composition 11

7 Exact startup and the halting-equivalence proof 12

8 Reproduction and additional experiments 13

8.1 Completed checks 13

8.2 A standalone command 13

8.3 Limited unsuccessful searches 14

9 Further research questions 14

A Exact identities and package organization 15

B Complete patch relative to the common U212 15

C Complete dense transition table 17

1 Result and exact scope

Let A denote the common incoming U212 table, R the new reassociated and entry-repaired 212-state intermediate, and M the final U209 table. The primary executable artifact is `machine/u209.tm`. Its conventions are:

Working states	$0, 1, \ldots, 208$
Separate halt label	209
Alphabet, blank	$\{0, 1\}$, blank 0
Tape and head	One two-way infinite tape; initial head coordinate 0
Initial configuration	State 0 and all tape cells 0
Transitions	Exactly 418 total rows; each moves left or right
Initialization	Included in the 209 working states

A row $q\ b\ w\ D\ r$ writes w, moves in direction D, and enters r. A transition into the halt label halts after its write and move. The native table retains sparse historical names, with native halt 254. Native 211 is a working helper; it must not be confused with a state count or with the halt label of an older dense table. A checked bijection relates native and dense tables.

Theorem 1.1 (New machine reduction)**.** *The source, startup, and literal-runtime arguments in this report establish the PA-formalizable equivalence*

$$\mathsf{PA} \vdash \mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Halt}(A, \mathrm{blank}).$$

For the final fold stage alone, R and M have equivalent halting behavior from state 0 on every common initial tape. Each step of R expands to between one and five steps of M.

The all-tape statement and time bound concern $R \to M$. They do not apply across $A \to R$, where source addresses change. For that stage, we use identical ordinal source behavior, complete canonical-frame runtime contracts, and the exact common blank startup.

The inherited proof-search and interpretation arguments identify $\neg\,\mathrm{Halt}(A, \mathrm{blank})$ with $\mathrm{Con}(\mathsf{U})$ and $\mathrm{Con}(\mathsf{PA})$ in PA. Their composition with Theorem 1.1 gives

$$\mathsf{PA} \vdash \neg\,\mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{U}) \leftrightarrow \mathrm{Con}(\mathsf{PA}). \tag{1}$$

Here "inherited" identifies the source of mathematical arguments; it is not an assumption that PA is consistent. Those arguments are retained and audited below, but have not been newly formalized in a proof-assistant kernel.

Layer	Evidence supplied or retained
Exact 209-state object	Complete tables, totality/reachability census, state bijection, fixed reconstruction and hashes.
New machine reduction	Explicit local proofs, exact source/control comparison, fresh unbounded symbolic runtime, exact blank entry.
Full consistency endpoint	Composition with the preserved numerical calculus, source compiler, safe startup, and U/PA proof translations.
Proof-assistant status	No new end-to-end Lean, Rocq, MM0, or exported PA certificate for this U209 table.

2 What the three supplied proposals actually contain

2.1 Pinned inventory and common parent

The comparison uses the four ZIP files in `docs/incoming` at ProveIt commit[1]:

`da4ab9dd68ccfc47ac852118601eaf1fdae53795`.

They are the unary theory archive and the Contextual, Research, and Rotated U211 archives [1, 2, 3, 4]. Their original archive hashes and pinned URLs are in `comparison/input_archives.json`.

The three U211 native tables are different, and a canonical breadth-first renaming shows that their rooted, action-labeled transition graphs are pairwise nonisomorphic. This is stronger than merely observing different file hashes. Nevertheless, after resolving addresses to ordinal instruction numbers, their entire effect/control graphs coincide. They share 3,909 primitives, 1,175 decrement occurrences, 197 distinct syntax subtrees, the same startup, and the same numerical proof search.

Their common U212 native table has SHA-256

`386ce16259d4c97e4af31b3ba494feaff11a0a1a066e0235dddc154b4cea38c4`.

The source identity is

`7b6539dc67b66ff73f22554018d7af5985d85574460a73fea5f025103c523d00`.

2.2 Machine and evidence comparison

	Contextual	Research	Rotated
Working states	211	211	211
Closed rotations	6	9	11
Deleted native state	$180 \to 163$	$181 \to 220$	$218 \to 157$
Mechanism	Predecessor tag; helper 211	Predecessor tag; helper 129	Predecessor zero; existing 157 behavior
Extra engine repair	$171/0 \to 0\,L\,205$	None	None
Maximum address depth	42	42	41
Maximum zero run	6	6	6
Fresh universal cases	5,084	5,084	5,084
Fresh row coverage	422/422	422/422	422/422
Symbolic inferences	1,277,525	1,280,108	1,245,944

All three full replays were rerun successfully for this investigation, in separate working copies. Fresh receipts are retained under `evidence/`. The symbolic-inference counts describe the verification work, not the actual running time of the proof search on blank tape.

The *Contextual* packet is particularly explicit about the two-cell selector proof and the arithmetic entry at native 171. Its independent finite executor is useful corroborating evidence. It also retains the fullest readable ancestry of the logical calibration.

The *Research/Decoder Reassociation* packet has the clearest standalone calibration chapter. It carefully distinguishes unary U from the numerical theory actually searched and distinguishes source reassociation

[1] https://github.com/VladimirReshetnikov/ProveIt/tree/da4ab9dd68ccfc47ac852118601eaf1fdae53795/docs/incoming

from the stronger all-tape theorem for the final fold. Its backward-entry guard addresses a genuine hazard of repurposing a shared decoder state.

The *Rotated* packet gives the simplest final deletion: two predecessor-zero cases replace state 218 by existing state 157. Its lower symbolic-inference count and maximum depth are useful engineering observations, but do not establish a faster complete consistency search.

2.3 A concrete documentation difference

Research and Rotated preserve a compact U212 dependency archive. Its U222 selection contains source-site and mapping data, but omits the original U222 report and all 23 retained logical reference files. That is an auditability limitation, not a counterexample to either machine.

Contextual's `baseline/selected_U212.zip`, of 10,243,682 bytes, retains the complete nested research chain U212, U213, U214, U218, U222. The latter contains the numerical-calibration paper, the exceptional-code correction, enumerator notes, and the logical templates. We retain that archive unchanged and separately include the exact user-specified unary archive. The selected U212 archive's SHA-256 is

`b5a5d19531f49b05aaad4e1b0f86d6f0de96c37b40a3617522b0d920cf83d6c6`.

The retained Rocq reference files have some unbundled imports. They are inspectable proof material, not a runnable new U-machine formalization.

There is consequently no single best packet for every purpose: Research is the strongest short calibration exposition, Rotated has the smallest local deletion argument, and Contextual provides the most complete research lineage. The new package combines these strengths.

3 Logical calibration retained by the new machine

3.1 Unary U and its PA strength

The primitive language of U is equality-free classical first-order logic over a nonempty domain with one binary relation, written membership. Put

$$x \subseteq a \quad :\Longleftrightarrow \quad \forall t\,(t \in x \to t \in a).$$

For every formula $\varphi(x)$ with no other free variable, the schema is

$$\forall a\,\exists b\,\left[\forall x\,(x \in b \to \varphi(x)) \;\wedge\; \forall x\,(x \subseteq a \to (\varphi(x) \to x \in b))\right]. \tag{2}$$

Bound variables are freshened; in particular a is not a free matrix parameter. This is a weak cover, not an assertion of extensionality or exact arbitrary-source power filtering.

The upper interpretation in the supplied report sends membership to binary bit membership. PA proves that a code whose one-bits are included among those of a is at most a. Formula-specific finite comprehension then constructs the required bit code by PA induction. This argument is performed one formula at a time and does not require a uniform truth predicate.

The lower interpretation's order of construction matters. It defines numerical representatives modulo coextensionality, with saturation supplying the membership congruence needed for quotient reasoning. Local predecessor facts yield induction *before* total successors are assumed. Comparison and least

numerical failures follow. Globally least failed numerical tuples are then bound inside legal unary matrices, yielding bounded numerical comprehension. Successors, numerical powers, saturated pair and graph codes, and canonical histories follow in that order. Pointwise history uniqueness uses the already available induction with ambient parameters; old graph objects are bound in the comprehension formulas. Addition, multiplication, and the translated PA induction schema result.

The fixed interpretations and schema-specific proof templates act by primitive-recursive operations on finite syntax. Induction on formulas and proof lengths gives the retained PA-provable finite-refutation translations. This supplies the internal assertion $\mathsf{PA} \vdash \mathrm{Con}(\mathsf{U}) \leftrightarrow \mathrm{Con}(\mathsf{PA})$, rather than only an external model-equiconsistency statement. Our independent reading found no concrete circularity or illicit free parameter in the examined argument. The archive does not implement a general arbitrary-PA proof translator, and the finite diagnostics do not establish the unbounded interpretation theorem.

3.2 The calculus actually executed

The literal machine does not parse the unary-schema restriction. Its inherited numerical calculus is calibrated to unrestricted weak covering W with additional matrix parameters, together with global irreflexivity IR. Unary U embeds into this theory, while the same bit-membership interpretation works in PA for arbitrary matrix parameters and proves irreflexivity. Combined with the lower interpretation into U, this gives PA-verifiable refutation reductions among the presentations.

The numerical coding uses dual Cantor pairing P with constructors $I = 4P$, $R = 4P + 1$, $Q = 4P + 2$, and an auxiliary equality constructor $4P + 3$. Code 0 is a closed truth alias; code 1 is diagonal membership used as falsity under irreflexivity. Seven logical rows, a dominated propositional row, modus ponens, generalization, and the gated covering/vacuity bundle form the retained system. It is distinct from the positive dyadic stream in the unary archive. That separate stream is not a newly compiled component of U209.

The U222 material retains the arguments for the small-code aliases, recovery of the propositional basis, auxiliary equality by two-sided membership indiscernibility, and finite quotation/refutation transport. Its old provisional warning about code zero is explicitly superseded by `references/exceptional_codes.txt`; the final paper uses the corrected unshifted construction. The correction must accompany the older notes.

The role-a covering emitter preserved by the later reductions uses matrix code and output label a, source label b, and tested label c, with $c \neq a, b$. That gate also justifies the atomic-vacuity conjunct. For positive a, effective matrix labels are smaller than a, so the output label is fresh. If $a = b$, the captured local subset guard becomes an emptiness condition under irreflexivity; an empty-source cover proves the exceptional instance. Completeness pads the desired matrix to a code $n > 1$, selects $(a, b, c) = (n, 0, 1)$, projects the cover, removes padding, and restores variable names. The new reduction changes none of this.

3.3 Relation to the supplied ZFC formalizations

The m254 development verifies the exact 254-state machine against its calculus and ZF/ZFC in Rocq. The ProveIt K215 development checks external Lean and Rocq endpoints for its exact table, and the separate k215-pa repository provides a Metamath Zero proof in PA for the K215/ZFC statement [6, 7, 8]. These results establish useful models for the required layers of verification. We read their current descriptions but did not rebuild those large formal projects. None of their endpoint certificates certifies the distinct U209 table or replaces the U/PA calibration above.

4 The new common source arrangement

4.1 Why simply combining the published patches fails

The three savings refer to different states, but the helper states are shared parts of the same decoder. A common arrangement for the first two folds still leaves a predecessor-one entrance to state 218. A naive rotation cascade removes that entrance while changing helper 129, which invalidates the second fold. State-count savings are therefore not additive without a compatible source allocation and a new runtime proof.

The successful construction expands the region being reassociated to native root 162. Its source interval is $[640, 3736)$, containing 3,096 primitives. The 18 internal selectors are pure: scanning bit b writes b, moves right, and chooses the corresponding child. The entire external-entry inventory is

$$156/1 \longmapsto 1\,R\,162.$$

There are no external entries to the other 17 selectors in that region.

Stopping expansion at closed factors gives the ordered word

$$(207, 67, 153, 132, 155, 159, 147, 176, 98, 218,\\ 178, 147, 146, 218, 176, 90, 211, 218, 116).$$

The corresponding lengths are

$$(503, 401, 56, 1042, 80, 162, 52, 80, 26, 56, 78, 52, 108, 56, 80, 27, 103, 56, 78).$$

Their sum is 3,096. Every factor's source boundary summary is $(0, 0, \text{false})$: no branch or return escapes and it contains no halt. Repeated factor names denote identical syntax trees, not just equal lengths. Closed sequential factors may be reassociated while retaining their ordered effects. The independent control-graph comparison below additionally checks every individual control destination.

4.2 The fixed 18-selector allocation

For a pair (a, b) in the following table, selector q has rows $q/0 \mapsto 0\,R\,a$ and $q/1 \mapsto 1\,R\,b$ before folding.

q	zero child	one child	q	zero child	one child
162	207	203	129	158	204
203	67	223	158	218	178
223	153	188	204	147	214
188	132	177	214	146	216
177	155	89	216	220	217
89	185	174	220	218	176
185	159	181	217	163	215
181	147	176	163	90	211
174	98	129	215	218	116

This is a new allocation relative to all three incoming packets. In particular, native 215 now encodes the final pair $(218, 116)$, rather than occupying an interior helper-sensitive position. Helpers 129 and 211 keep the branches needed by the first two folds. The final occurrence of 218 is reached through a zero edge. This is the essential compatibility step.

Three further closed rotations, written as (outer, inner, right-child), are

$$(206, 199, 1), \quad (205, 206, 1), \quad (7, 205, 1).$$

They reshape the closed sequence

$$(86, (164, (186, (180, 166)))) \quad \longmapsto \quad ((86, (164, (186, 180))), 166).$$

Only the association is significant in this display. The executable construction and complete source graphs settle the precise binary syntax. These small rotations are inside the enlarged 3,096-primitive region, although they are separate proof steps from the 18-selector reassociation. They make both remaining entrances to 180 write one immediately before moving right.

4.3 The arithmetic-entry repair

Native 7 also has an arithmetic back-entry: the original row $171/0 \mapsto 0\,L\,7$. After the rotations, it must become

$$171/0 \longmapsto 0\,L\,205.$$

The complete backward-entry census into repurposed states finds this sole old entrance. For the two exhaustive possibilities for the cell immediately left of state 171's head, the old and new paths agree:

Left bit	Old path	New path	Common endpoint
0	$171 \to 7 \to 86$	$171 \to 205 \to 86$	State 86, original head position
1	$171 \to 7 \to 205 \to 164$	$171 \to 205 \to 206 \to 164$	State 164, one cell right

The current cell is zero. Both paths retain both cells, read no exterior cell, and take respectively two or three steps. In the second case the return reads the zero just written by state 171. Omitting the repair is rejected by the local checker.

4.4 Exact finite source evidence

The final source retains all 3,909 opcodes and operands in the same order; exactly 3,096 addresses change. Two independently implemented successor resolvers agree on all 5,083 control edges: one uses intervals of subtree leaves and outward ancestor walking, the other uses binary increment/cut and zero-padded decoding. The latter checks that a successor has no stray one beyond its next instruction leaf.

There remain 197 distinct syntax subtrees, maximum address length 42, and maximum consecutive-zero run six. The existing seven-zero origin detector therefore retains the required address bound. Independent literal prefix derivations check all 3,908 internal source prefixes, without overwriting bits or reading an unspecified exterior cell. Their maximum length is 49 steps in the new intermediate and final tables, versus 50 in the parent. These structural checks complement the unbounded primitive simulation; they do not replace it.

5 Three state deletions with complete local proofs

5.1 The predecessor invariants

In R, the complete incoming inventories are:

State	Incoming row keys	Each row writes/moves
163	217/0	$0, R$
180	132/1, 199/1	$1, R$
220	216/0	$0, R$
181	185/1	$1, R$
218	158/0, 211/0, 215/0, 220/0	$0, R$

Hence every entry to 163, 220, or 218 has a zero immediately left of the head, and every entry to 180 or 181 has a one there. These are consequences of the last literal transition on *any* tape. They are not invariants inferred from a finite blank run. The initial state 0 is outside this set.

The original selectors at these entries are

$$163 : (90, 211), \quad 180 : (152, 211), \quad 220 : (218, 176), \quad 181 : (147, 176), \quad 218 : (90, 86).$$

Define

$$\chi(180) = 163, \quad \chi(181) = 220, \quad \chi(218) = 157, \qquad \chi(q) = q \text{ otherwise.}$$

Redirect every incoming target by χ, delete the outgoing rows of 180, 181 and 218, and replace two retained rows by

$$163/0 \mapsto 0\,L\,211, \qquad 220/0 \mapsto 0\,L\,129. \tag{3}$$

No state is added. All 414 rows outside $\{163, 180, 220, 181, 218\} \times \{0, 1\}$ satisfy the literal quotient equation

$$\delta_M(\chi(q), b) = (w, D, \chi(z)) \quad \text{whenever} \quad \delta_R(q, b) = (w, D, z).$$

5.2 The two tagged folds

Before deleting 218, the first pair uses helper 211 to inspect the preceding tag on read zero. Tag zero selects $211 \to 218 \to 90$; tag one selects $211 \to 184 \to 152$. The second pair uses helper 129: tag zero selects $129 \to 158 \to 218$; tag one selects $129 \to 204 \to 147$. Each path restores the scanned cells and reaches the original target one cell to the right. The read-one branches already coincide in each pair and take one step.

This first stage deletes 180 and 181 and yields the retained `machine/doublefold210.native.tm`. At that point the complete incoming inventory for 218 has only 158/0, 211/0, and 215/0, all writing zero and moving right. The old 220/0 entrance has become the backward selector in (3); its helper path reaches 218 via 158/0.

5.3 Deleting 218 by an existing behavior

With zero in the predecessor cell, existing state 157 implements both branches of state 218 in three steps:

$$\begin{aligned} \text{read } 0: &\quad 157 \xrightarrow{0L} 67 \xrightarrow{0R} 151 \xrightarrow{0R} 90, \\ \text{read } 1: &\quad 157 \xrightarrow{1L} 148 \xrightarrow{0R} 186 \xrightarrow{1R} 86. \end{aligned}$$

The predecessor zero and current bit are preserved. Both paths end one cell right of the old head, with every exterior cell unread. This proves the final $218 \to 157$ substitution with the complete incoming invariant.

5.4 A direct certificate for the combined result

Composing the deletions changes a helper inside the first fold. In particular, the old 163/read-zero step now takes five steps, not three. The exact direct local paths are shown below. "Tag" is the bit at head offset -1; the current scanned bit is at offset 0. Every endpoint is one cell right, with the whole tape unchanged.

Old	New	Tag	Read	New state path, including endpoint	Steps
163	163	0	0	$163 \to 211 \to 157 \to 67 \to 151 \to 90$	5
163	163	0	1	$163 \to 211$	1
180	163	1	0	$163 \to 211 \to 184 \to 152$	3
180	163	1	1	$163 \to 211$	1
220	220	0	0	$220 \to 129 \to 158 \to 157$	3
220	220	0	1	$220 \to 176$	1
181	220	1	0	$220 \to 129 \to 204 \to 147$	3
181	220	1	1	$220 \to 176$	1
218	157	0	0	$157 \to 67 \to 151 \to 90$	3
218	157	0	1	$157 \to 148 \to 186 \to 86$	3

Lemma 5.1 (Direct fold simulation)**.** *For every common initial tape and head position, runs of R and M from state 0 have the same halting behavior. At matched boundaries the tape and head agree, and the new state is χ of the old state. A finite old run of N steps expands to a new run of N' steps with $N \le N' \le 5N$.*

Proof. At the initial boundary $\chi(0) = 0$. At an ordinary state the quotient equation supplies one new step with the required endpoint. At a special state, the complete incoming census proves its predecessor tag; the ten displayed cases supply the matching finite block. Each block is positive, has length at most five, preserves the entire tape and head at its endpoint, and contains no internal halt. Induction on the old run concatenates the blocks and gives the bounds. Determinism forces a new run from a boundary to traverse exactly the indicated block. A new halt therefore occurs only at a matching old halt. Equivalently, an arbitrarily long nonhalting old prefix supplies an at least equally long nonhalting new prefix. This proves both directions. All arguments are finite syntax and natural-number induction and can be expressed in PA. □

The independent local checker represents only the two specified cells and rejects any attempted exterior read. It verifies all ten paths, all 414 ordinary rows, the complete tag inventories, and the two engine-repair paths. Thirteen corrupted controls are rejected. It gives identical results with normal Python and `-O`; its conditions are explicit exceptions.

6 Fresh unbounded literal-runtime verification

6.1 The quantified whole-tape frame

Put $B(v) = 0\,1^{v+1}$ and $C(v) = 1\,0\,1^{v}$. For a finite list A, use $B(A)$ and $C(A)$ for concatenations. At a primitive boundary the control is native 84, and the head points to the beginning of address p in

$$\cdots 0\,1\;p\;0^{56+g-|p|}\;1\;B(v_0)\cdots B(v_8)B(a_0)B(A)\;0\cdots . \tag{4}$$

Here $g, a_0, v_0, \ldots, v_8$ are arbitrary naturals and A is an arbitrary finite list, possibly empty. Thus the full spare list is nonempty. Logical registers x_i occupy physical slots $\pi(i)$, where

$$\pi = (6, 2, 0, 4, 3, 7, 8, 5, 1), \qquad v_{\pi(i)} = x_i.$$

Each decrement is split into $x_i = 0$ and $x_i = u + 1$. Both infinite zero exteriors are represented. This is a family of complete tapes, not a test sample of bounded registers.

Run lengths, time, and displacement are represented by 13-component affine expressions: a constant, the gap, nine primary values, the first spare value, and the actual width of $B(A)$. The final component denotes the width of the same represented list, not an unrelated free length. Coefficientwise nonnegativity supplies the inequalities used by the checker.

6.2 Sound elementary proof rules

The two inference modules are byte-identical to the retained U212/Contextual modules. Their rules admit elementary proofs:

1. *Literal row.* A transition is read from the final table. A run may be read one cell at a time only when its affine length has constant at least one and nonnegative coefficients. The symbolic update is precisely that row's write, move, and control change for every valuation.

2. *Unary scan.* A self-loop moving in the current direction can consume an entire homogeneous run of length n. Induction on n, including zero, shows that it abbreviates exactly n literal steps, with displacement $\pm n$ and the recorded output run. It places no upper bound on n.

3. *Finite-list traversal.* Before using a list law, the checker rederives its one-block case from the final table with arbitrary v and opaque contexts. Native entries 64, 0, 40, and 97 supply rightward $B \to B$, rightward $B \to C$, leftward reversed $B \to B$, and leftward reversed $C \to B$ traversals. Each returns to its entry state after $v + 2$ steps. The empty list and concatenation of the checked block prove the law for every finite list.

4. *Word identities.* Adjacent equal-symbol runs are joined; zero-length runs disappear; a finite zero suffix is absorbed by the infinite blank exterior. The identity $C(A)1 = 1B(A)$ and its reversal follow first for one block, then by induction on A. These transformations preserve the complete represented word and its position relative to the head.

An opaque-context read, an unjustified possibly-empty run read, or a missing traversal law causes failure. The proof-search cap of 4,000 symbolic inferences per contract limits the checker, not the register values or list lengths covered by a successful derivation.

6.3 Primitive endpoints and source composition

For increment, positive decrement, zero decrement, control branch, control return, and halt, the expected endpoint is independently specified by the source instruction. Increment appends a spare block with value one; positive decrement increases the extra gap by one. Binary-address overflow and the associated head displacement are explicit. For a nonhalting instruction the final control, entire tape, and head must equal the expected frame. Halt must reach native 254. The duration is affine with nonnegative coefficients and strictly positive constant term.

A raw successor address need not itself be a leaf address. The independent binary resolver proves that zero padding decodes to the specified next leaf and that no stray one remains after that leaf.

Consequently the raw endpoint and the next canonical source frame describe the same tape. This is the bridge from the primitive contracts to the ordinal source graph.

The fresh release replay derives

$$3909 + 1175 = \boxed{5084}$$

guarded contracts, using **1,250,049 symbolic inference applications**. It covers all **418 literal rows**, rederives all four list-block laws, and has minimum duration constant **5**. Its case data matches the separately generated discovery run exactly. A bijective internal relabeling allows use of the unchanged legacy engine; saved case rows are converted back to native labels. The table and source hashes bind both runs.

These are executable ordinary inductive derivations, not proof-assistant kernel objects. Transition coverage is an additional diagnostic; the quantified contracts and soundness of the four rules carry the argument.

7 Exact startup and the halting-equivalence proof

A separate sparse-tape execution checks all 48,140 blank-start transitions. The entire transition trace and complete tape equal the parent's. No edited or deleted row occurs in that prefix. The endpoint is native 84, head -2, header coordinate 54, and source ordinal 136, with address `001100110101010`. In logical order $(N, T, W, A, D, H, X, Y, L)$ the registers are

$$(1, 0, 13, 0, 0, 1, 0, 0, 1).$$

There are 43 spare registers of value one and 120 one-cells in total. An independent frame constructor checks every one-cell and the zero exterior.

The inherited startup is an interior AX continuation, not an ordinary clean source root. It corresponds to 141 source steps from ordinal 106 with store $(1, 0, 4, 0, 0, 1, 0, 0, 1)$; this finite source bridge is rerun as well. Cached value one at that point is an arithmetic selector parameter, not a theorem of falsity. The retained suffix proof produces nine theorem heads above opaque tail 4 and returns safely to candidate one.

That tail would be unsafe if exposed: $4 = P(1, 1)$ and $1 = P(1, 0)$. The retained scheduler credit proof gives maximum debt two from candidate one, leaving margin seven under the nine initial heads. It tracks an explicit ghost-list length because $P(0, 0) = 0$ prevents recovery of a unique stack length from its integer code. Strong compilation works above arbitrary old stacks after resetting the relevant counters; successful candidate indices are cofinal and every unsuccessful candidate computation terminates. These facts supply both soundness and completeness despite the unusual initial continuation. They are preserved, not reproved by bounded blank simulation.

Proof of Theorem 1.1. The retained parent runtime and the newly derived final runtime implement their respective sources on the same quantified frame class. Both machines reach the same fully specified blank-start frame and same source ordinal. The source comparison identifies every instruction effect and every control successor, so induction on primitive steps relates the two runs, translating their address strings but preserving logical registers and the corresponding frame parameters. Every primitive takes finite positive time. Thus a source halt yields a finite literal halt in either implementation, and a literal halt occurs only in the matching source halt contract. The identical nonhalting startup prefixes complete the blank-run equivalence. The retained parent obligations and the new obligations use finite syntax, unary/list induction, and proof-length induction; their composition is PA-formalizable. Lemma 5.1 gives the separate all-tape result for the final stage. □

Applying the numerical-calculus and finite-refutation translations identified in Section 3 then gives (1). This proof uses no assumption of Con(PA) and proves no unconditional nonhalting statement.

8 Reproduction and additional experiments

8.1 Completed checks

Check	Result
Final working states / rows	209 / 418
Changed retained rows / deleted rows of U212	37 / 6
Preserved source primitives / control edges	3,909 / 5,083
Changed source addresses	3,096
Internal prefix interfaces	3,908
Direct fold / arithmetic-repair cases	10 / 2
Ordinary quotient rows	414
Rejected corrupted controls	13
Fresh arbitrary-value runtime contracts	5,084
Symbolic inference applications	1,250,049
Arbitrary-list block lemmas	4
Literal runtime row coverage	418 / 418
Minimum affine duration constant	5
Exact common blank prefix	48,140 steps
Independent finite primitive cases	15,252
Finite primitive literal steps	6,773,382

The C++ executor is independent of the symbolic induction engine. It checks three finite assignments for each primitive branch, including nonzero gaps and different spare lists. A separate two-million-step blank prefix also completed without halting. These are regression checks only; neither their length nor success supports a conclusion of consistency.

The unary-theory diagnostics were freshly rerun: six suites in normal Python and under `-O`, all 16 named target certificates, and 4,096 bounded descriptions in each mode, with release hashes checked before and after. Those checks audit implementations and finite examples, not the unbounded interpretation theorem.

8.2 A standalone command

From the extracted top-level directory, with Python 3.10 or later:

```
python3 -B reproduce.py
```

No network, solver, compiler, or proof-assistant installation is needed. The default command verifies all release hashes, binds parent data and the unchanged induction engine to the preserved U212 ZIP, reconstructs exact machine/source bytes in a temporary directory, checks local paths both normally and under `-O`, checks source and startup, compares the three incoming machines, and freshly rederives every runtime case. The symbolic engine requires assertions and rejects optimized Python. The release itself is unchanged. The preparation environment took about 79 seconds for the final runtime phase.

```
python3 -B reproduce.py --quick
```

This explicitly skips the fresh unbounded runtime derivation. It is useful for construction review, but is not equivalent to the default full replay. An optional `-report` path must be outside the sealed release. The C++ regression command and PDF rebuild instructions are in `README.txt`. PDF bytes can vary across TeX versions and timestamps; machine and source outputs are deterministic.

8.3 Limited unsuccessful searches

After obtaining U209, an additional screen examined 43,264 ordered direct state aliases, allowing at most 15 replacement steps for one old step. Another examined 7,038 opposite-predecessor-tag helper folds within a specified one-row modification class, allowing at most 17 replacement steps. Neither produced an additional accepted certificate. Their scripts and receipts are retained in `research/`.

Both searches are intentionally narrow and incomplete. Finite assignments screen candidates before a bounded exhaustive local branch check; the screen can reject sound equivalences outside its step bound or interface class. These negative results give no lower bound on state count. The positive U209 result rests on the separate complete local and runtime arguments, not on the search procedure.

9 Further research questions

1. **Can the local proof be the first kernel-checked component?** The 414 ordinary quotient rows, ten two-cell contracts, and two repair contracts form a small independent target for Lean, Rocq, or MM0. Formalizing the general positive-block simulation lemma would make later folds cheap to certify, while preserving the separate source-reassociation obligation.

2. **Can helper requirements be compiled into an allocation search?** The successful move of state 215 shows that closed-sequence association should optimize decoder states subject to explicit helper equations, incoming tags, engine back-entries, address length, and zero-run bounds. An exhaustive or SAT-based allocator could search this finite design space and export a small checked recipe. The two-cell equations are much cheaper to test than a full runtime replay, but the latter remains a final gate.

3. **Can the origin detector be shortened soundly?** The current source has valid addresses with six consecutive zeros, including its first instruction. Simply deleting another origin-detection state is unsound. A candidate would need a source/control transformation or alternate delimiter invariant eliminating the obstruction everywhere, followed by fresh whole-tape contracts and startup calibration.

4. **Can the full U/PA translation be made executable and formal?** The current mathematical interpretation is stronger evidence than finite diagnostics, but not an exported general proof translator. A fixed syntax translation with explicit PA proof templates would close that audit gap. The old Rocq excerpts need their import closure and a new U-specific endpoint; the existing K215/ZFC theorem cannot be reused as that endpoint unchanged.

5. **Is a cheaper fresh-binder interface available?** The covering emitter's gate also supplies logical vacuity. Deriving the required arbitrary fresh-binder membership-vacuity family from a cheaper automatically fresh family could remove code. Without that derivation, dropping a gate test or its conjunct is a hypothesis, not a state reduction.

6. **Can proof search, rather than decoder allocation, be simplified?** All three U211 packets and U209 retain the same ordinal computation. The next substantial improvement may require a different formula encoding, proof compiler, or startup. Such work must address aliases, finite-cutoff completeness, and the protected tail anew. A smaller axiom schema alone does not determine the compiled binary state count.

The precise achievement here is the compatible combination of three previously conflicting machine reductions, with an explicit 209-state table and fresh evidence at the stated proof boundary. It establishes a concrete next formalization target and a reproducible upper-bound candidate.

A Exact identities and package organization

Dense U209 table SHA-256
`5ded628d692ff442e4346c61eb3db3c77b0ac2c80c1ecced45cbfc483be40e08`

Native U209 table SHA-256
`d0a155d3c683789f3bf5327c0cfb2a0cfd78076af8f1654d4ca426ccbb604d16`

Final source SHA-256
`cddbed7f20e7a8b085b7ca0e76811d07b021522375643b5d1134eec0d2d23164`

Reassociated prefold U212 table SHA-256
`1ca076433b5f4be053477fd2552eea9faa6542bb2405e33e43c46ced5e165c7a`

Two-fold intermediate table SHA-256
`b1ccdd7ccb14fe4427ea5f39a39e24b266d597e25055be26c45b1228fcd8b5e7`

Hashes identify bytes, not semantic truth. The independent checks consume the identified tables and source. `SHA256SUMS` seals all delivered payloads other than itself.

Path	Contents
`report.tex`, `tables/`	Complete editable report and generated table fragments.
`machine/`	Final dense/native tables, both intermediates, source, state bijection, fixed construction receipt.
`code/`, `reproduce.py`	Deterministic reconstruction, independent local/source/startup checking, unchanged induction engine, runtime replay.
`baseline/`	Frozen U212 files, exact unary archive, and full nested research dependency archive.
`comparison/`	The three input table/source snapshots and exact archive provenance.
`evidence/`	Fresh checks, all 5,084 symbolic cases, independent mathematical reviews, and comparison receipts.
`research/`	Scope-limited unsuccessful further searches.
`licenses/`, `NOTICE.txt`	Retained contributor attribution, license texts and provenance.

B Complete patch relative to the common U212

Native labels are used in this appendix. Each displayed action is write, direction, target. The table lists every changed retained row and all six deleted rows; all other rows are unchanged. The deterministic builder also checks the final source and state renaming.

Native state	Read	U212 action	U209 action
7	0	0R86	0R205
7	1	1R205	1R166
89	0	0R155	0R185
129	1	1R178	1R204
132	1	1R180	1R163
158	0	0R98	0R157
158	1	1R218	1R178
162	0	0R203	0R207
162	1	1R116	1R203
163	0	0R211	0L211
163	1	1R218	1R211
171	0	0L7	0L205
174	0	0R159	0R98
174	1	1R185	1R129
177	0	0R132	0R155
180	0	0R152	*deleted*
180	1	1R211	*deleted*
181	0	0R176	*deleted*
181	1	1R129	*deleted*
185	0	0R217	0R159
185	1	1R215	1R220
188	0	0R153	0R132
199	0	0R180	0R186
199	1	1R166	1R163
203	0	0R207	0R67
204	0	0R216	0R147
204	1	1R163	1R214
205	0	0R164	0R86
206	0	0R186	0R164
211	0	0R218	0R157
214	0	0R147	0R146
214	1	1R146	1R216
215	0	0R214	0R157
215	1	1R204	1R116
216	0	0R218	0R220
216	1	1R220	1R217
217	0	0R147	0R163
217	1	1R181	1R215
218	0	0R90	*deleted*
218	1	1R86	*deleted*
220	0	0R176	0L129
220	1	1R90	1R176
223	0	0R67	0R153

C Complete dense transition table

The following table contains both transitions for every working state. Halt is 209 and has no outgoing rows. A cell wDq means write w, move D, enter q. State 0 is initial on the blank tape.

q	read 0	read 1	q	read 0	read 1	q	read 0	read 1
0	1R92	1R0	70	0L89	1R78	140	0R96	1R159
1	0L1	1L2	71	0R55	1L79	141	0R175	1R166
2	0L3	1L2	72	0R80	1R81	142	0R12	1R159
3	0L5	1L2	73	1R0	1R63	143	0R88	1L166
4	1R6	0L4	74	0R74	1R82	144	0R88	1R158
5	0L9	1L2	75	0L145	1L141	145	0L152	1L163
6	0R6	0R10	76	0R85	1R86	146	0R84	1R88
7	0R190	1R159	77	0R20	1R38	147	0R159	1R12
8	0R14	1R13	78	0R89	1L132	148	0R83	1R143
9	0L15	1L2	79	0L108	1R56	149	0R144	1R155
10	0L16	0R10	80	0R49	0R33	150	0L66	1L141
11	0L26	1R128	81	0R91	0L42	151	0R150	1R170
12	0R19	1R20	82	0R178	1R77	152	0R159	1R114
13	0L89	1L31	83	0R96	1R12	153	0R97	1R97
14	0L11	0R22	84	0R88	1R97	154	0R49	0R32
15	0L23	1L2	85	0R207	1R21	155	0R192	1R188
16	1L1	0L1	86	0R116	1R100	156	0L195	1R195
17	0R24	0R32	87	0R174	1R167	157	0L187	1R96
18	0R98	1R113	88	0R102	1R103	158	0R168	1R179
19	0R28	1R29	89	0R80	1R79	159	0R19	1R171
20	0R30	1R31	90	0R101	1R196	160	0R164	1R187
21	0R24	0R59	91	0R105	1R106	161	0R159	1R96
22	0R33	1R22	92	0R65	0R0	162	0R125	1R164
23	0L34	1L2	93	0R27	1R107	163	0R97	1R162
24	0L35	0R0	94	0R183	0L208	164	0L190	1R142
25	0R36	1R25	95	1L109	1L95	165	0L66	1R173
26	0R37	1R38	96	0R19	1R110	166	0R146	1R172
27	0R39	0L40	97	0R60	1R60	167	0R96	1R123
28	0R41	1R42	98	0R68	1R135	168	0R161	1R143
29	0R43	1R44	99	0R93	1R108	169	0R148	1R87
30	0R45	1R46	100	0R105	0L208	170	0R161	1R96
31	0R21	1R47	101	0R117	1R118	171	0R181	1R122
32	0R48	1R32	102	0R71	1R58	172	0L193	1R60
33	0R32	1R33	103	0R57	1R119	173	0R145	1R88
34	0L74	1L2	104	0R21	1R120	174	0R152	1R202
35	1R35	1L1	105	0R21	0L10	175	0R140	1R84
36	0R50	1R36	106	0R61	1R42	176	0R175	1R75
37	0R17	1R111	107	0R121	1R8	177	0R126	1R169
38	0R13	1R52	108	0R101	1R122	178	0R138	1R116
39	0L11	0R36	109	0L208	0L95	179	0R97	1R51
40	0L4	1L53	110	0R124	1R104	180	0R168	1R51
41	0R24	0R25	111	0R115	1L72	181	0R17	1L131
42	0R13	0L16	112	0R165	1R181	182	0R143	1R149
43	0R55	1R56	113	0R160	1R184	183	0R76	1R116
44	0R57	1R58	114	0R148	1R146	184	0R106	1R124
45	0R24	0R54	115	0R11	1R129	185	1L40	1L109
46	0R60	1R61	116	0L131	0L1	186	0R175	1R156
47	0R62	0L16	117	0R39	1R72	187	0R182	1R183

q	read 0	read 1	q	read 0	read 1	q	read 0	read 1
48	0R25	1R48	118	0R61	1R132	188	0R66	1R205
49	0L35	0R63	119	0R61	1R133	189	0R140	1R198
50	0R16	1R50	120	0R70	0L16	190	0R84	1R191
51	0R147	1R143	121	0R14	1R132	191	0R157	1R186
52	0R67	1R68	122	0R21	1R134	192	0R64	1R7
53	0L40	1L53	123	0R151	1R189	193	0R150	1R127
54	0R22	1R54	124	0R137	1L20	194	0R18	1R94
55	0L11	0R50	125	0R130	1R180	195	0R150	1R173
56	0R60	0L16	126	0R204	1R156	196	0R104	1R96
57	0R69	1R70	127	0R75	1R176	197	0R90	1R88
58	0R61	1R71	128	0R75	1R206	198	0R139	1R200
59	0R54	1R59	129	0R203	1R157	199	0R150	1R112
60	0R49	0R25	130	0R12	1R141	200	0R202	1R201
61	0L28	1L72	131	0R30	1R69	201	0R156	1R199
62	0L117	1L30	132	0R39	1L124	202	0L123	1R168
63	0R73	1R63	133	0R153	0L16	203	0R197	1R28
64	0R97	1R150	134	0R154	0L16	204	0R163	1R193
65	1R185	1L53	135	0R88	1R136	205	0R146	1R177
66	0R144	1R145	136	0R197	1R140	206	0R161	1R194
67	0R76	1R209	137	0R24	0R48	207	0R45	1R100
68	0L197	1R31	138	0R99	1R37	208	0L4	0L208
69	0R41	1R62	139	0R172	1R148			

References

[1] *Unary Power Covering at PA Strength*, 27 October 2026. User-specified `pa_unary_power_cover.zip`, retained exactly under `baseline/`. Original pinned file: https://github.com/VladimirReshetnikov/ProveIt/blob/da4ab9dd68ccfc47ac852118601eaf1fdae53795/docs/incoming/pa_unary_power_cover.zip.

[2] *U211: a 211-state candidate for the consistency of Peano arithmetic*, Contextual packet, 28 October 2026. `U211_ConPA_Contextual_2026-10-28.zip`, at the same pinned incoming directory.

[3] *U211: a decoder-sharing candidate for Con(PA)*, Research/Decoder Reassociation packet, 28 October 2026. `U211_ConPA_Research_2026-10-28.zip`, at the same pinned directory.

[4] *U211: decoder reassociation and a guarded state deletion*, Rotated packet, 28 October 2026. `U211_ConPA_Rotated_2026-10-28.zip`, at the same pinned directory.

[5] *U212 ConPA Stacked*, 28 October 2026, retained as `baseline/selected_U212.zip`; contains the nested U213, U214, U218 and U222 research dependencies and original notices.

[6] Jeff's busy-beaver project, *A verified 254-state Turing machine that halts iff ZFC is inconsistent*, machine credited to Mira-Cult. https://github.com/jeff87654/busy-beaver/tree/zf-254/ZF/m254.

[7] ProveIt, *Formal correctness of the exact 215-state machine*. https://github.com/VladimirReshetnikov/ProveIt/tree/main/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215.

[8] Elliot Glazer, *K215 halts iff ZFC is inconsistent (Metamath Zero, PA)*. https://github.com/elliotglazer/k215-pa.

[9] Elliot Glazer, *K230: the theory K*. https://github.com/elliotglazer/k230-rocq#the-theory-k.